Department of Computer Science

Course Code: CIC201

Course Title: Artificial Intelligence

Fall 2025

Lab 11

Introduction to Neural Networks

Objective:

Objective: The Neural Network Basics Lab aims to impart a foundational understanding of neural networks, covering key concepts such as architecture, activation functions, and the implementation of a simple perceptron in Python using NumPy. Through hands-on tasks, participants will enhance their skills in modifying activation functions, handling multi-feature perceptrons, introducing bias, and applying neural networks to real-world datasets, fostering a practical understanding of neural network fundamentals.

Name of Student	
Student ID	
Date of Lab Conducted	
Marks Obtained	
Remarks	
Signature	

UIT University
Department of Computer Science
CIC201 – Artificial Intelligence
Spring 2024

1. Objective

Objective: The Neural Network Basics Lab aims to impart a foundational understanding of neural networks, covering key concepts such as architecture, activation functions, and the implementation of a simple perceptron in Python using NumPy. Through hands-on tasks, participants will enhance their skills in modifying activation functions, handling multi-feature perceptrons, introducing bias, and applying neural networks to real-world datasets, fostering a practical understanding of neural network fundamentals..

2. Instructions

You have to perform the following tasks yourselves. Raise your hand if you face any difficulty in understanding and solving these tasks. Plagiarism is an abhorrent practice and you should not engage in it.

3. How to Submit

Submit lab work using TEAMS

Neural networks: Neural networks are computational models inspired by the human brain. They consist of interconnected nodes or neurons that process information. Applications include image and speech recognition, natural language processing, and various machine learning tasks.

Basic architecture: neurons, layers, and connections:

Neural networks are composed of layers, including an input layer, one or more hidden layers, and an output layer. Neurons in one layer are connected to neurons in the next layer, and each connection has a weight. During training, these weights are adjusted to improve the network's performance.

Neurons:

A neuron is the basic building block of a neural network. It is inspired by the biological neurons in the human brain. In a neural network, a neuron receives input signals, processes them, and produces an output. The basic structure of a neuron includes:

Input Weights (Synaptic Weights):

Each input to a neuron is associated with a weight. These weights represent the strength of the connections between the neuron's input and its output. The neuron multiplies each input by its corresponding weight and sums up these weighted inputs.

Summation Function:

The weighted inputs are summed together.

Activation Function:

The result of the summation is then passed through an activation function. The activation function introduces non-linearity to the system, allowing the neural network to learn complex patterns. Common activation functions include Sigmoid, Hyperbolic Tangent (tanh), Rectified Linear Unit (ReLU), etc.

Output:

The output of the neuron is the result of the activation function applied to the sum of weighted inputs.

Layers:

A neural network is composed of layers of neurons. There are typically three types of layers:
Input Layer: The first layer of the network that receives the initial input. Each neuron in the input layer represents a feature of the input data.

Hidden Layers: Layers between the input and output layers are called hidden layers. Deep neural networks have multiple hidden layers. Each neuron in a hidden layer receives inputs from the previous layer, processes them, and passes the result to the next layer.

Output Layer: The final layer of the network that produces the output. The number of neurons in the output layer depends on the nature of the task

Connections:

Connections represent the flow of information between neurons in different layers. Each connection has an associated weight, and it transmits the output of one neuron to the input of another. During training, the network learns the optimal weights for these connections through a process called backpropagation.

Feedforward and backpropagation concepts:

Feedforward is the process of passing data through the network from the input layer to the output layer. Backpropagation is a supervised learning algorithm that involves adjusting the weights in the network based on the difference between the predicted output and the actual output. This process helps the network learn and improve its accuracy.
This forward pass, where the input data moves through the network to produce an output, is called the feedforward process.

Backpropagation is the mechanism by which the network learns from its mistakes and adjusts its weights and biases to improve performance.

Activation functions (sigmoid, tanh, ReLU, etc.):

Activation functions introduce non-linearity to the network. Common activation functions include sigmoid (for binary classification), tanh (similar to sigmoid but with a range of -1 to 1), and ReLU (Rectified Linear Unit, which allows the network to learn complex patterns).

Build a simple perceptron in Python using NumPy:

```
# Import NumPy
import numpy as np

# Define the perceptron function
def perceptron(input_data, weights):
    # Calculate the weighted sum
    weighted_sum = np.dot(input_data, weights)

    # Apply a simple step function as activation
    output = 1 if weighted_sum > 0 else 0

    return output

# Sample data
input_data = np.array([1, 2])
weights = np.array([0.5, -0.5])

# Test the perceptron
output = perceptron(input_data, weights)
print("Perceptron Output:", output)
```

```
Perceptron Output: 0
```

weighted_sum = np.dot([1, 2], [0.5, -0.5]) = 1 * 0.5 + 2 * (-0.5) = 0.5 - 1.0 = -0.5

The activation function (step function) introduces non-linearity because the decision to output 0 or 1 is based on a non-linear threshold.

Build a simple perceptron in Python generating random data:

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt

# Define the perceptron function
def perceptron(input_data, weights):
    # Calculate the weighted sum
    weighted_sum = np.dot(input_data, weights)

    # Apply a simple step function as activation
    output = 1 if weighted_sum > 0 else 0

    return output

# Generate random data for a binary classification task
np.random.seed(42)
num_samples = 10
#np.random.rand generates random numbers between 0 and 1.
features = np.random.rand(num_samples, 2) * 2 - 1  # Random values between -1 and 1
weights = np.array([0.5, -0.5])

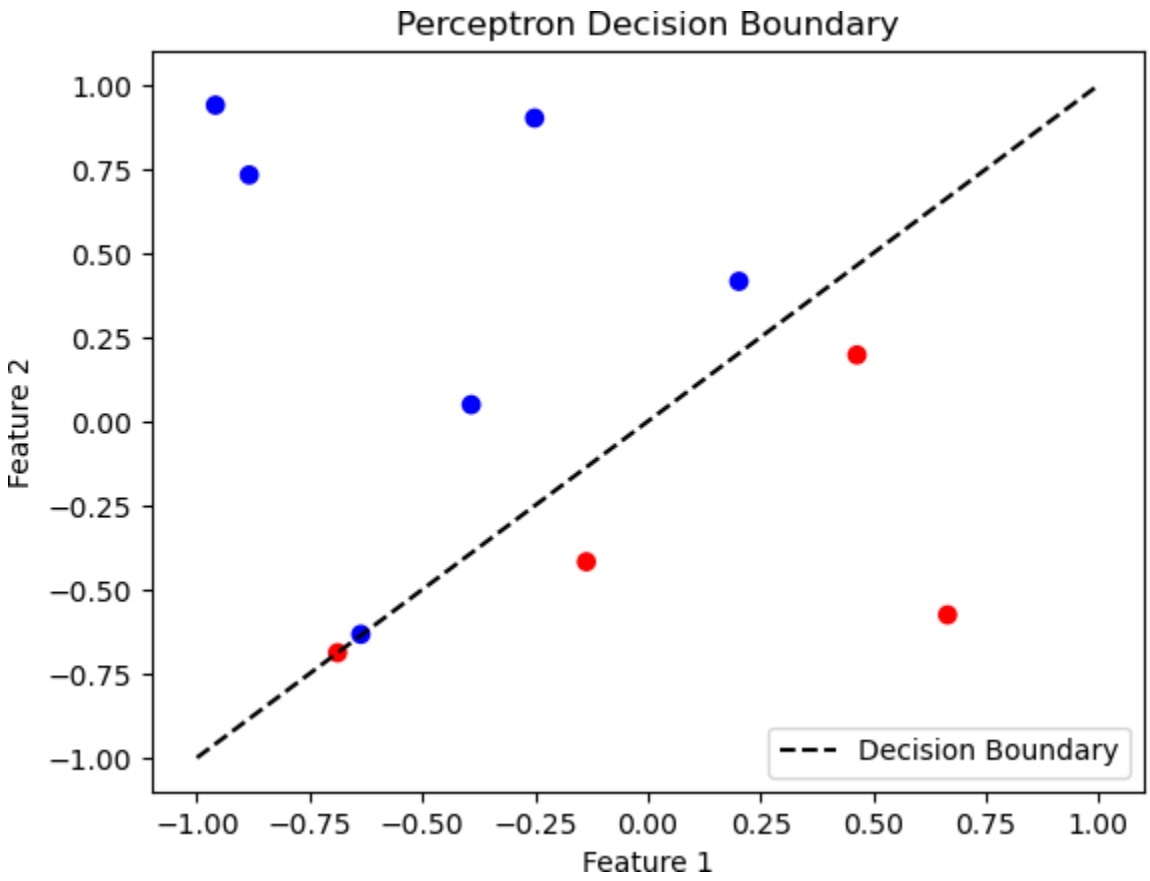
# Classify data using the perceptron
# Applies the perceptron to each data point in features and collects the predictions.
predictions = [perceptron(sample, weights) for sample in features]

# Visualize the data and decision boundary
plt.scatter(features[:, 0], features[:, 1], c=predictions, cmap='bwr')
plt.title('Perceptron Decision Boundary')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

# Plot the decision boundary (line defined by the perceptron)
x_line = np.linspace(-1, 1, 10)
y_line = -weights[0]/weights[1] * x_line
plt.plot(x_line, y_line, color='black', linestyle='--', label='Decision Boundary')

plt.legend()
plt.show()
```

Output:



Lab Tasks¶

Task 1: Expand the Perceptron Function
Modify the perceptron function to use a different activation function, such as the sigmoid function or the hyperbolic tangent (tanh) function. Update the hands-on code to test the modified perceptron function with the new activation function. Compare the results and discuss the differences in the decision boundaries.

Task 2: Multi-feature Perceptron Extend the perceptron function to handle more than two features. Update the sample data accordingly and test the modified perceptron function with the new dataset. Visualize the decision boundary in a 3D plot or use color to represent the classification in a scatter plot.

Task 3: Introduce Bias Modify the perceptron function to include a bias term. Update the sample data and weights accordingly. Discuss how the introduction of a bias term affects the decision boundary and the perceptron's ability to capture patterns in the data.

Bais

"bias" refers to a learnable parameter that provides an additional degree of freedom to the model. Bias plays a crucial role in enhancing the flexibility and expressive power of neural networks, allowing them to learn complex patterns and relationships in data

Mathematical Notation:

Mathematical Notation: In a neural network, each neuron or node has an associated bias term. For a neuron with bias b and input vector x, the total input to the neuron is given by

$y=mx+b$

Learnable Parameter:

Learnable Parameter: Bias is a learnable parameter that the neural network adjusts during the training process to minimize the difference between predicted and actual outputs.

Role of Bias:

Shifting Activation: The bias term allows the activation function to shift horizontally, influencing when the neuron becomes active.

Handling Non-Centered Data: Bias is particularly useful when dealing with data that is not centered around zero. It enables the model to adapt to the offset in the data.

Learning Bias during Training:

Gradient Descent: During training, the neural network learns the optimal values for bias by adjusting them iteratively using gradient descent optimization algorithms.

Bias Update Rule: The update rule for bias in gradient descent involves computing the gradient of the loss with respect to the bias and adjusting it in the opposite direction to minimize the loss.

Lab Tasks

Task 4: Real-world Data Classification Use a real-world dataset (e.g., from sklearn.datasets) and adapt the perceptron code to perform binary classification on the dataset. Visualize the data points and the decision boundary. Discuss the challenges and limitations of using a simple perceptron for real-world datasets.

Task 5: Compare with sklearn Perceptron Explore the Perceptron implementation available in the scikit-learn library (sklearn.linear_model.Perceptron). Compare its performance and decision boundary with your custom perceptron implementation. Discuss any differences observed.